

RESTful Schema-Aware Data Integration & Transformation API

PBCST304 Project Report

Submitted by:

**Aaron Paul
Athuljith S
Dijith Dinesh
Peter Joe Menachery
Pranav Manoj**

**MDL24CSBS002
MDL24CSBS021
MDL24CSBS023
MDL24CSBS048
MDL24CSBS049**



Department of Computer Engineering
Govt. Model Engineering College, Thrikkakara
Ernakulam, Kochi 682021
<https://www.mec.ac.in>

October 2025

Model Engineering College, Thrikkakara
Department of Computer Engineering



CERTIFICATE

This is to certify that the project report entitled: **“RESTful Schema-Aware Data Integration & Transformation API”** is a bonafide record of the work carried out by

Aaron Paul

MDL24CSBS002

Athuljith S

MDL24CSBS021

Dijith Dinesh

MDL24CSBS023

Peter Joe Menachery

MDL24CSBS048

Pranav Manoj

MDL24CSBS049

Third semester B.Tech Computer Science & Engineering & Business Systems

students, in partial fulfillment of the requirements for the course **PBCST304: Object Oriented Programming** for the award of the degree of Bachelor of Technology from **APJ Abdul Kalam Technological University**.

Raheena Salihin

Project Guide

Assistant Professor

Dept. of Computer Engineering

Dr. Binu V P

Head of the Department

Associate Professor

Dept. of Computer Engineering

October 22, 2025

Acknowledgements

We wish to extend our profound gratitude to the individuals whose guidance and support were instrumental in the successful completion of this project.

Our deepest appreciation is reserved for our project guide, *Raheena Salihin*. Her technical acumen and rigorous feedback were pivotal in transforming our initial concept into a robust and functional application. Her mentorship consistently challenged us to think critically, adhere to the highest standards of software engineering, and navigate the complexities of system architecture.

We are also grateful to our Head of Department, *Dr. Binu V P*, and the entire faculty of the Department of Computer Engineering for fostering an academic environment that encourages innovation and for providing the administrative support essential for our work.

Finally, we thank our peers for their collaborative spirit and our families for their enduring patience and support throughout this endeavor.

Aaron Paul MDL24CSBS002

Athuljith S MDL24CSBS021

Dijith Dinesh MDL24CSBS023

Peter Joe Menachery MDL24CSBS048

Pranav Manoj MDL24CSBS049

Abstract

In modern software development, client applications are often tightly coupled to the database schema of backend services, leading to brittle and difficult-to-maintain systems. Any change in the backend schema can cause cascading failures, necessitating costly update cycles for all client applications. This project addresses this critical issue by designing and implementing a **RESTful Schema-Aware Data Integration and Transformation API**.

This system acts as an intelligent, universal data gateway that completely decouples client applications from the physical database. It provides a robust abstraction layer that allows clients to fetch, filter, sort, and transform data using a simple, intuitive URL-based query language, abstracting away the complexities of SQL. The API supports on-the-fly data transformation into multiple formats (JSON, CSV, XML) and dynamic field renaming through a powerful view-management system.

Built from the ground up in Java without external frameworks, the project is a case study in applying core Object-Oriented principles—Encapsulation, Inheritance, Polymorphism, and Abstraction—to build a modular, maintainable, and secure backend service. The system enforces security through a role-based access control (RBAC) mechanism using API keys, providing a centralized and managed access point to data. The final product is a lightweight, powerful, and flexible tool for any developer needing to expose data securely and efficiently.

Contents

Contents	i
List of Figures	iv
List of Tables	v
1 Introduction	1
1.1 Proposed Project	1
1.1.1 Problem Statement	1
1.1.2 Proposed Solution	2
2 Software Requirement Specification (SRS)	4
2.1 Introduction	4
2.1.1 Purpose	4
2.1.2 Intended Audience	4
2.1.3 Project Scope	5
2.2 Overall Description	6
2.2.1 Product Perspective	6
2.2.2 Product Functions	6
2.2.3 User Classes and Characteristics	6
2.2.4 Operating Environment	7
2.2.5 Design and Implementation Constraints	7
2.3 Hardware and Software Requirements	7
2.3.1 Hardware Requirements	7
2.3.2 Software Requirements	8
2.4 Functional Requirements	8
2.5 Non-functional Requirements	9
3 System Design	11
3.1 System Architecture	11

3.2	Use Case Diagram	13
3.3	Class Diagram	14
3.4	Activity Diagram	15
3.4.1	Data Request Workflow (GET)	15
3.4.2	Data Modification Workflow (POST/PUT/DELETE)	15
4	Data Flow Diagram (DFD)	18
4.1	Level 0 DFD (Context Diagram)	18
4.2	Level 1 DFD	19
5	Implementation	20
5.1	Core Algorithms and Logic	20
5.1.1	Secure Query Parsing (ApiQueryParser)	20
5.1.2	Polymorphic Data Export ('Exporter' Module)	21
5.2	Database Schema and Core Tables	21
5.3	Development Tools	22
6	API Reference and Usage Demonstration	24
6.1	Setup and First Run	24
6.2	Core Concepts	25
6.2.1	Authentication	25
6.3	API Endpoint Reference	25
6.3.1	Data Endpoint: /data/{tableName}	25
6.3.2	Schema Endpoint: /schema/{tableName}	25
6.3.3	View Endpoint: /view/{viewName}	25
6.3.4	Admin Endpoint: /admin/execute	26
6.4	Query Parameter Reference	26
6.5	Usage Demonstration and Outputs	26
6.5.1	Server Startup and Live Logging	26
6.5.2	Data Modification (POST Request)	26
6.5.3	On-the-Fly Data Transformation	27
7	Testing	30
7.1	Testing Methodologies	30
7.2	Unit Testing	31
7.2.1	Secure Query Parameter Parsing	31
7.2.2	Data Exportation Logic	31
7.3	Integration Testing	31
7.3.1	Handler to Database Layer Integration	31
7.3.2	View Mapping and Data Translation Integration	32

7.4	System Testing	32
7.4.1	End-to-End Scenario Testing	32
8	Results	34
9	Conclusion	35
10	Future Scope	36
10.1	Relational Data Support (JOINS)	36
10.2	Performance Caching	36
10.3	Containerized Deployment (Docker)	36
10.4	Administrative GUI	37
10.5	Expanded Database Support	37
	References	38

List of Figures

3.1	Layered System Architecture	11
3.2	System Use Case Diagram	14
3.3	System Class Diagram	15
3.4	Activity Diagram: Data Request (GET)	16
3.5	Activity Diagram: Data Modification	17
4.1	Level 0 Data Flow Diagram (Context Diagram)	18
4.2	Level 1 Data Flow Diagram	19
6.1	Server Startup and Real-time Request Logging	27
6.2	Creating a New Record via a POST Request	27
6.3	Default Data Output in JSON Format	28
6.4	Data Transformed into CSV Format	28
6.5	Data Rendered as an HTML Table	29
6.6	Data Formatted as a Markdown Table	29

List of Tables

5.1	Schema for the <code>api_keys</code> table	22
5.2	Schema for the <code>api_views</code> table	22
5.3	Schema for the <code>table_metadata</code> table	22

Chapter 1

Introduction

1.1 Proposed Project

This document outlines the design, architecture, and implementation of the RESTful Schema-Aware Data Integration and Transformation API. The project's core mission is to solve a pervasive problem in software architecture: the tight coupling between client applications and backend data sources. By providing a flexible and intelligent middleware, this project aims to streamline development, enhance system resilience, and improve data security.

1.1.1 Problem Statement

In contemporary application development, client-side software (such as web frontends, mobile apps, or other backend services) is frequently designed to consume data directly from a backend database. This direct or near-direct interaction leads to several significant problems:

- **Brittleness and High Maintenance Overhead:** Client applications often contain logic that is explicitly tied to the database schema. If a database administrator renames a column (e.g., from `user_email` to `email_address`), every client application that references the old column name will break. This necessitates a coordinated, and often costly, update and redeployment of all dependent applications.
- **Developer Inefficiency:** Client-side developers are forced to write repetitive boilerplate code to fetch, parse, and transform data into the specific format required by their application's views or models. This introduces redundancy and distracts from the core task of building user-facing features.
- **Lack of a Single Source of Truth:** When data logic is scattered across

multiple client applications, there is no centralized point of control for data access rules, validation, or transformation. This leads to inconsistencies and makes the system difficult to manage.

- **Security Risks:** Exposing database structures, even indirectly, can create security vulnerabilities. Furthermore, managing fine-grained access control across numerous disparate clients is a complex and error-prone task.

These issues collectively lead to systems that are difficult to evolve, expensive to maintain, and insecure.

1.1.2 Proposed Solution

To address the challenges outlined above, we propose the development of a standalone, universal API server that acts as an abstraction layer between clients and the physical database. This **RESTful Schema-Aware Data Gateway** provides a comprehensive solution:

- **Decoupling through Abstraction:** The API completely decouples clients from the database. Clients interact with a stable, well-defined RESTful interface, while the API server is responsible for translating these requests into the appropriate database queries. If the database schema changes, only the API's internal mapping needs to be updated, leaving all client applications unaffected.
- **Brittleness and High Maintenance Overhead:** Client applications often contain logic that is explicitly tied to the database schema. If a database administrator renames a column (e.g., from `user_email` to `email_address`), every client application that references the old column name will break. This necessitates a coordinated, and often costly, update and redeployment of all dependent applications.
- **On-the-Fly Data Transformation:** The API can deliver data in a variety of formats (JSON, CSV, XML, etc.) based on a simple URL parameter (`?format=csv`). It also features a powerful "view" system that allows for dynamic renaming of data fields, enabling clients to receive data in the exact structure they require without any backend code changes.
- **Centralized Security and Control:** The API enforces a Role-Based Access Control (RBAC) model using API keys. Permissions (read-only, write, admin) are managed centrally, providing a single, secure, and auditable point of access to the data.

This approach establishes a single centralized source of truth for data access, significantly reduces the client-side development effort, and creates a more secure, resilient, and maintainable system architecture.

Chapter 2

Software Requirement Specification (SRS)

2.1 Introduction

This chapter provides a detailed specification of the requirements for the RESTful Schema-Aware Data Integration API. It outlines the functional and non-functional requirements, constraints, and system attributes, serving as the primary reference for the design, development, and testing phases of the project.

2.1.1 Purpose

The purpose of this Software Requirement Specification (SRS) document is to provide a clear, comprehensive, and unambiguous description of the system's intended behavior and capabilities. It aims to:

- Establish a formal agreement between the development team and stakeholders on what the software will and will not do.
- Serve as the foundational blueprint for all subsequent project activities, including architectural design, implementation, and quality assurance.
- Provide a baseline for evaluating and verifying the final product against the initial requirements.

2.1.2 Intended Audience

This document is intended for a wide range of stakeholders, including:

- **Project Developers:** To understand the system's requirements and constraints, guiding their implementation efforts.
- **System Architects:** To design a robust and scalable architecture that meets the specified requirements.

- **Testers and QA Engineers:** To create test plans and test cases that ensure the system behaves as expected.
- **Project Managers:** To manage the project scope and track progress against the defined requirements.
- **End-Users (Client Developers):** To understand the capabilities and limitations of the API they will be consuming.

2.1.3 Project Scope

The project scope is to develop a standalone, general-purpose RESTful API server that can be placed in front of any relational database to provide a flexible and secure data access layer.

In Scope:

- Development of a runnable Java application that starts a lightweight HTTP server.
- Implementation of API endpoints for data querying (CRUD operations), schema discovery, and view management.
- A dynamic URL parser that can translate query parameters into secure SQL.
- A modular data export system for transforming data into multiple formats (JSON, CSV, XML, etc.).
- A role-based access control (RBAC) system based on API keys stored in the database.
- An administrative endpoint for executing a limited set of secure DDL commands.

Out of Scope:

- Development of any client-side applications (web UIs, mobile apps). The project focuses solely on the backend API server.
- User authentication systems (e.g., OAuth, JWT). The API handles authorization via its own keys, but assumes user authentication is managed by the client application.
- Support for NoSQL databases. The initial design is focused on JDBC-compliant relational databases.

- A graphical user interface (GUI) for administering the API. All administration is handled via API endpoints.

2.2 Overall Description

2.2.1 Product Perspective

The RESTful API is a self-contained, standalone middleware product. It is not an end-user application but a backend service intended to be used by other software systems. It operates independently of any specific client application or business domain, functioning as a generic and reusable data gateway.

2.2.2 Product Functions

The major functions of the system are summarized below:

1. **Dynamic Data Retrieval:** Allow authorized clients to fetch data from any table using a flexible set of URL parameters for filtering, sorting, pagination, and field selection.
2. **On-the-Fly Data Transformation:** Convert raw database results into various client-requested formats (e.g., JSON, CSV) and restructure the data's field names using predefined or ad-hoc views.
3. **Secure Data Modification:** Provide secure endpoints for creating, updating, and deleting records, subject to authorization rules.
4. **Schema Discovery:** Enable clients to programmatically query the schema of database tables, including column names and data types.
5. **Centralized Security:** Authenticate all requests via API keys and authorize actions based on the roles (read, write, admin) associated with each key.
6. **Administrative Control:** Offer a secure endpoint for administrators to perform basic database management tasks.

2.2.3 User Classes and Characteristics

Client Developer: The primary user of the API. This user is a software developer building a client application. They are expected to have a good understanding of RESTful principles and HTTP. They require a well-documented, intuitive, and powerful API to accelerate their development process.

System Administrator: This user is responsible for deploying, configuring, and maintaining the API server. They have a technical background in systems administration and database management. They require secure access to administrative functions to manage the database schema and API keys.

2.2.4 Operating Environment

The API server is designed to be platform-independent and will operate in the following environment:

- **Operating System:** Any modern 64-bit operating system that can run a Java Virtual Machine (JVM), including Windows, Linux, and macOS.
- **Runtime Environment:** Java Development Kit (JDK) 11 or newer.
- **Database:** The system will interface with any JDBC-compliant relational database. It is developed and tested with SQLite for portability but is designed to be easily configurable for databases like PostgreSQL, MySQL, etc.

2.2.5 Design and Implementation Constraints

- The system must be implemented in Java without the use of major external frameworks like Spring or Jakarta EE to ensure a lightweight footprint and focus on core OOP principles.
- All database queries generated from user input must be parameterized to prevent SQL injection vulnerabilities. Direct string concatenation of user input into SQL queries is strictly forbidden.
- The final deliverable must be a self-contained, runnable application (e.g., a JAR file) that can be easily deployed and started from the command line.
- The system's internal architecture must be modular and adhere to the principles of high cohesion and loose coupling.

2.3 Hardware and Software Requirements

2.3.1 Hardware Requirements

- **CPU:** 2-core x86-64 CPU, 2.0 GHz or faster.

- **RAM:** Minimum of 2 GB RAM. Recommended 4 GB or more for handling a large number of concurrent requests.
- **Storage:** Minimum of 500 MB of free disk space for the application, logs, and a small database.

2.3.2 Software Requirements

- **Runtime:** Java SE Development Kit (JDK) 11 or later.
- **Database Engine:** SQLite 3.x (for default operation) or a network-accessible instance of PostgreSQL, MySQL, etc.
- **Testing Client:** A command-line tool capable of making HTTP requests, such as ‘curl’.

2.4 Functional Requirements

This section details the functional requirements of the system, specified by feature.

FR-1: API Key Authentication

- **FR-1.1:** The system shall require an API key to be sent in the Authorization HTTP header for all requests to protected endpoints.
- **FR-1.2:** The system shall validate the provided API key against the `api_keys` table in the database.
- **FR-1.3:** Requests with missing or invalid API keys shall be rejected with a 401 Unauthorized status.

FR-2: Dynamic Data Retrieval (GET)

- **FR-2.1 Filtering (where):** The system shall allow clients to filter results based on column values using operators such as `eq`, `neq`, `gt`, `gte`, `lt`, `lte`, `like`, `contains`, and `in`.
- **FR-2.2 Sorting (sort):** The system shall allow clients to sort results by one or more columns in ascending (`asc`) or descending (`desc`) order.
- **FR-2.3 Pagination (limit, page):** The system shall support pagination of results.
- **FR-2.4 Field Selection (fields):** The system shall allow clients to specify a comma-separated list of columns to be included in the response.

FR-3: Data Modification (POST, PUT, DELETE)

- **FR-3.1 Create (POST):** An authorized client shall be able to create a new record in a table by sending a JSON object in the request body.
- **FR-3.2 Update (PUT/PATCH):** An authorized client shall be able to update an existing record by its ID.
- **FR-3.3 Delete (DELETE):** An authorized client shall be able to delete one or more records that match a where clause. Mass deletion without a where clause must be forbidden.

FR-4: Data Transformation

- **FR-4.1 Format Export (format):** The system shall be able to return data in multiple formats, including JSON (default), CSV, XML, HTML, and Markdown, based on the format URL parameter.
- **FR-4.2 View Management (view):** The system shall allow users to create, read, and use persistent views that map database column names to client-friendly aliases.
- **FR-4.3 Ad-hoc Renaming (replace):** The system shall support temporary, one-off renaming of fields in the output using the replace parameter.

FR-5: Schema Discovery

- **FR-5.1:** A client shall be able to retrieve the schema (column names and data types) of any accessible table via the /schema/{tableName} endpoint.

FR-6: Administrative Functions

- **FR-6.1:** A client with admin-level privileges shall be able to execute DDL commands (CREATE TABLE, DROP TABLE) via the /admin/execute endpoint.
- **FR-6.2:** The admin endpoint must validate incoming SQL to ensure it is a single, non-malicious command.

2.5 Non-functional Requirements

NFR-1: Performance

- **NFR-1.1 Response Time:** For simple queries on indexed columns, the API should have an average response time of less than 200ms under normal load.

- **NFR-1.2 Concurrency:** The system should be able to handle at least 50 concurrent requests without significant performance degradation.

NFR-2: Scalability

- The system's architecture should allow it to scale vertically (by adding more CPU/RAM) and be deployable in a horizontally scaled environment (behind a load balancer).

NFR-3: Security

- The system must be protected against SQL injection attacks by exclusively using parameterized queries.
- Access to write and admin functionalities must be strictly enforced by the RBAC mechanism.
- The system should not expose any sensitive information, such as database credentials or stack traces, in its error responses to the client.

NFR-4: Reliability

- The system should be reliable, with an expected uptime of 99.9%.
- The system must handle errors gracefully and return appropriate HTTP status codes and error messages. It should not crash due to invalid user input.

NFR-5: Usability (for Developers)

- The API endpoints and URL query parameters must be intuitive, consistent, and easy to use.
- Error messages returned by the API should be clear and helpful to a developer trying to debug an issue.

NFR-6: Maintainability

- The source code must be well-documented, with comments explaining complex logic.
- The project must be structured into logical, modular packages that follow the principles of high cohesion and loose coupling.

Chapter 3

System Design

3.1 System Architecture

The system is designed following a classic **Layered Architecture** pattern. This architectural style promotes a strong separation of concerns, enhances modularity, and improves the maintainability of the application. Each layer has a distinct responsibility and communicates only with the layers directly adjacent to it. The architecture is composed of four primary layers.

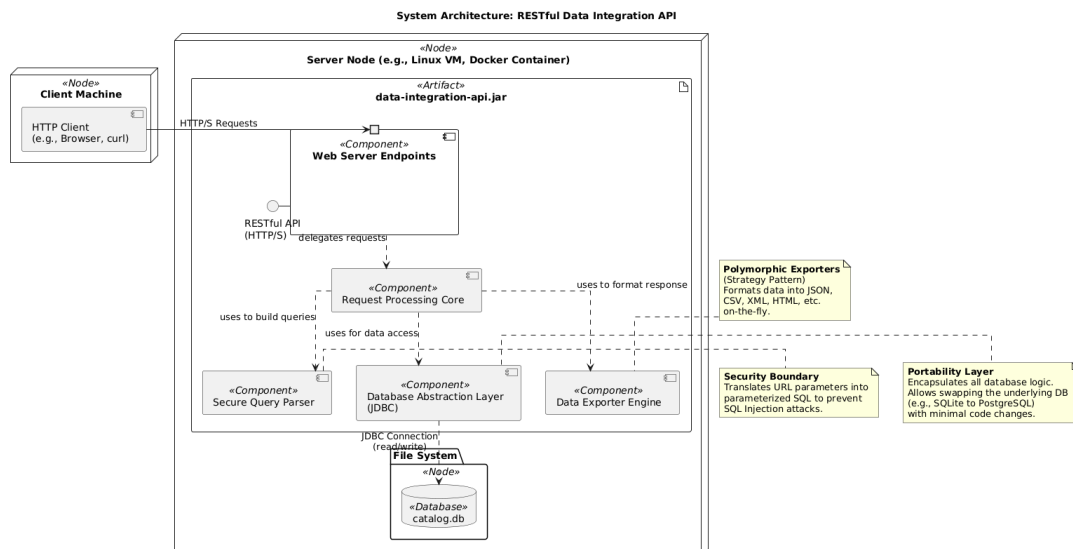


Figure 3.1: Layered System Architecture

Presentation Layer (API Endpoints)

This is the topmost layer and the public face of the application. It is the sole entry point for all client requests.

- **Components:** ‘WebServer’, ‘DataRequestHandler’, ‘SchemaRequestHandler’, ‘AdminRequestHandler’, ‘ViewCreateHandler’.
- **Responsibilities:**
 - Initializes and manages the lifecycle of the lightweight HTTP server.
 - Receives raw HTTP requests and parses their methods, headers, paths, and query parameters.
 - Routes each request to the appropriate handler based on its URL path (e.g., ‘/data’, ‘/schema’).
 - Delegates the processing of the request to the Service Layer.
 - Receives the final processed data from the Service Layer and formats it into an HTTP response, setting the correct status codes and headers before sending it back to the client.

Service Layer (Business Logic)

This layer contains the core application logic and orchestrates the fulfillment of a request. It is responsible for all decision-making and data transformation.

- **Components:** ‘ApiQueryParser’, ‘ViewMapper’, ‘ExporterEngine’ (a conceptual grouping of all ‘Exporter’ classes).
- **Responsibilities:**
 - Takes the parsed request data from the Presentation Layer.
 - Validates user input and permissions.
 - Uses the ‘ApiQueryParser’ to securely translate URL parameters into a parameterized SQL query object. This is a critical security function.
 - Uses the ‘ViewMapper’ to handle the translation of data fields between the client’s view and the database’s schema.
 - Coordinates with the Data Access Layer to execute the query.
 - Uses the ‘ExporterEngine’ (Strategy Pattern) to transform the raw data from the DAL into the client-requested format (JSON, CSV, etc.).

Data Access Layer (DAL)

This layer is the sole gateway to the database. It completely encapsulates all database interaction logic.

- **Component:** ‘DatabaseManager’.
- **Responsibilities:**
 - Manages the database connection pool.
 - Executes the query objects received from the Service Layer using JDBC.
 - Handles all aspects of ‘PreparedStatement’ creation, parameter binding, and execution.
 - Maps the ‘ResultSet’ from the database into a list of ‘GenericProduct’ domain objects.
 - Abstracts away all vendor-specific SQL or JDBC details, making the application portable across different database systems.

Persistence Layer

This layer represents the physical storage of the data.

- **Component:** The database itself (e.g., an SQLite file, a PostgreSQL server).
- **Responsibilities:**
 - Storing and retrieving data efficiently.
 - Ensuring data integrity and consistency through transactions and constraints.

3.2 Use Case Diagram

The Use Case diagram illustrates the high-level functional requirements of the system and how different actors (users) interact with it. It provides a clear overview of the system’s boundaries and its primary capabilities. The system has two main actors: the **Client Developer** and the **Administrator**, where the Administrator is a specialized type of developer with extended privileges.

Actor Descriptions:

Client Developer: Represents a programmer or another automated system that consumes the API to build applications. Their goal is to access and manipulate data in a simple and efficient manner.

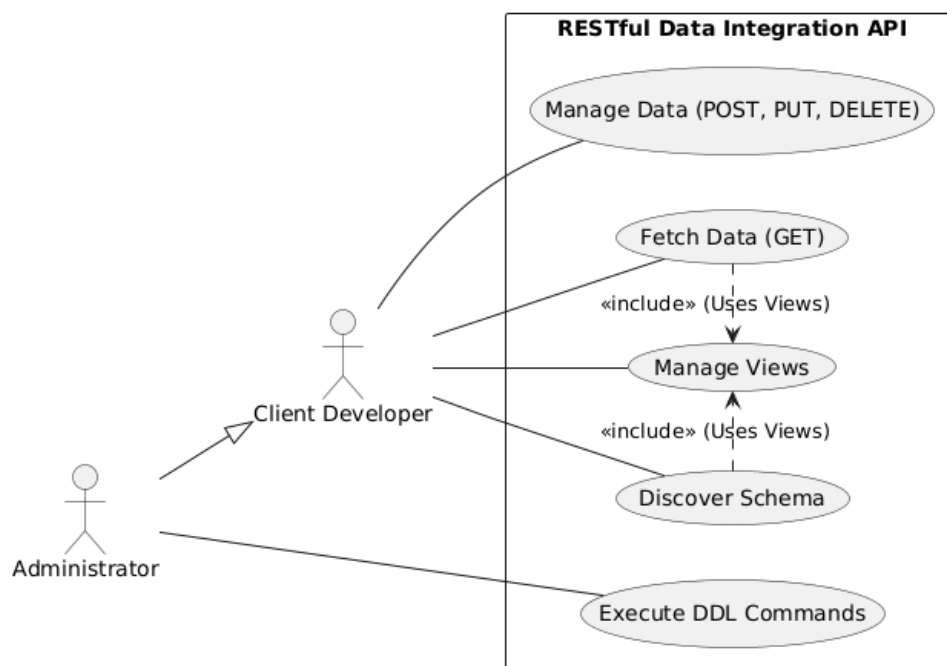


Figure 3.2: System Use Case Diagram

Administrator: A privileged user responsible for managing the underlying database schema and system-level configurations, such as API keys.

Use Case Descriptions:

Fetch Data (GET): Allows the developer to read data with complex filtering, sorting, and pagination.

Manage Data (POST, PUT, DELETE): Allows the developer to create, update, and delete records.

Discover Schema: Allows the developer to programmatically query the structure of data tables.

Manage Views: Allows the developer to create and manage aliases for data fields.

Execute DDL Commands: A specialized use case for the Administrator to manage the database schema directly via the API.

3.3 Class Diagram

The Class Diagram provides a static, structural view of the system's architecture. As a cornerstone of Object-Oriented Design, it is essential for visualizing the key classes, their attributes, methods, and the relationships between them. This

is particularly important for this project, given its emphasis on applying core OOP principles without reliance on external frameworks.

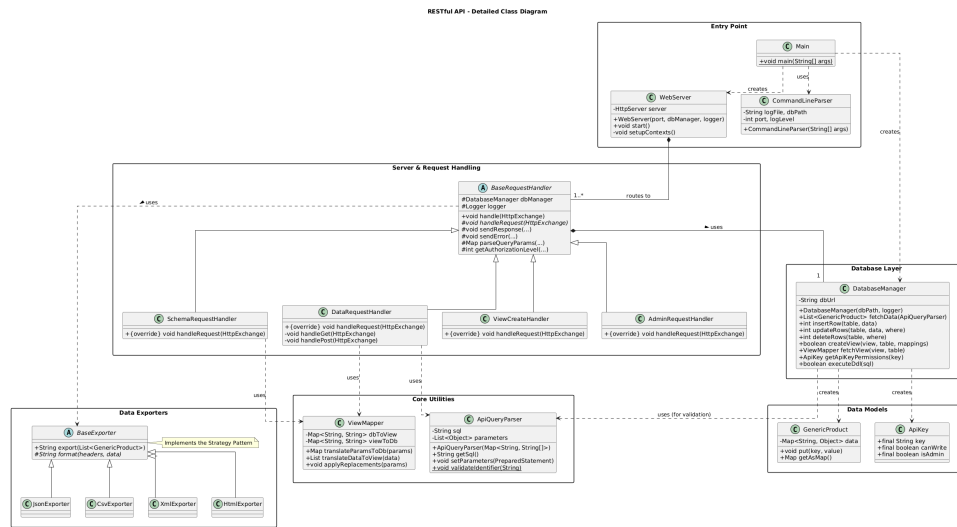


Figure 3.3: System Class Diagram

3.4 Activity Diagram

Activity diagrams visually represent the workflows of the system, illustrating the sequential flow of actions and decisions for a specific process. They provide a clear, step-by-step visualization of how users or system components engage to achieve a goal.

3.4.1 Data Request Workflow (GET)

This diagram (Figure 3.4) shows the flow for a read-only ‘GET’ request. It begins with receiving the request and proceeds through authorization, query parsing, data fetching, transformation, and finally, sending the response. It highlights the decision points for authorization and view handling.

3.4.2 Data Modification Workflow (POST/PUT/DELETE)

This diagram (Figure 3.5) details the workflow for write operations. It shows the initial authorization check for write permissions, followed by a branching logic that handles ‘POST’/‘PUT’ requests (which require JSON body parsing) differently from ‘DELETE’ requests (which require ‘where’ clause validation).

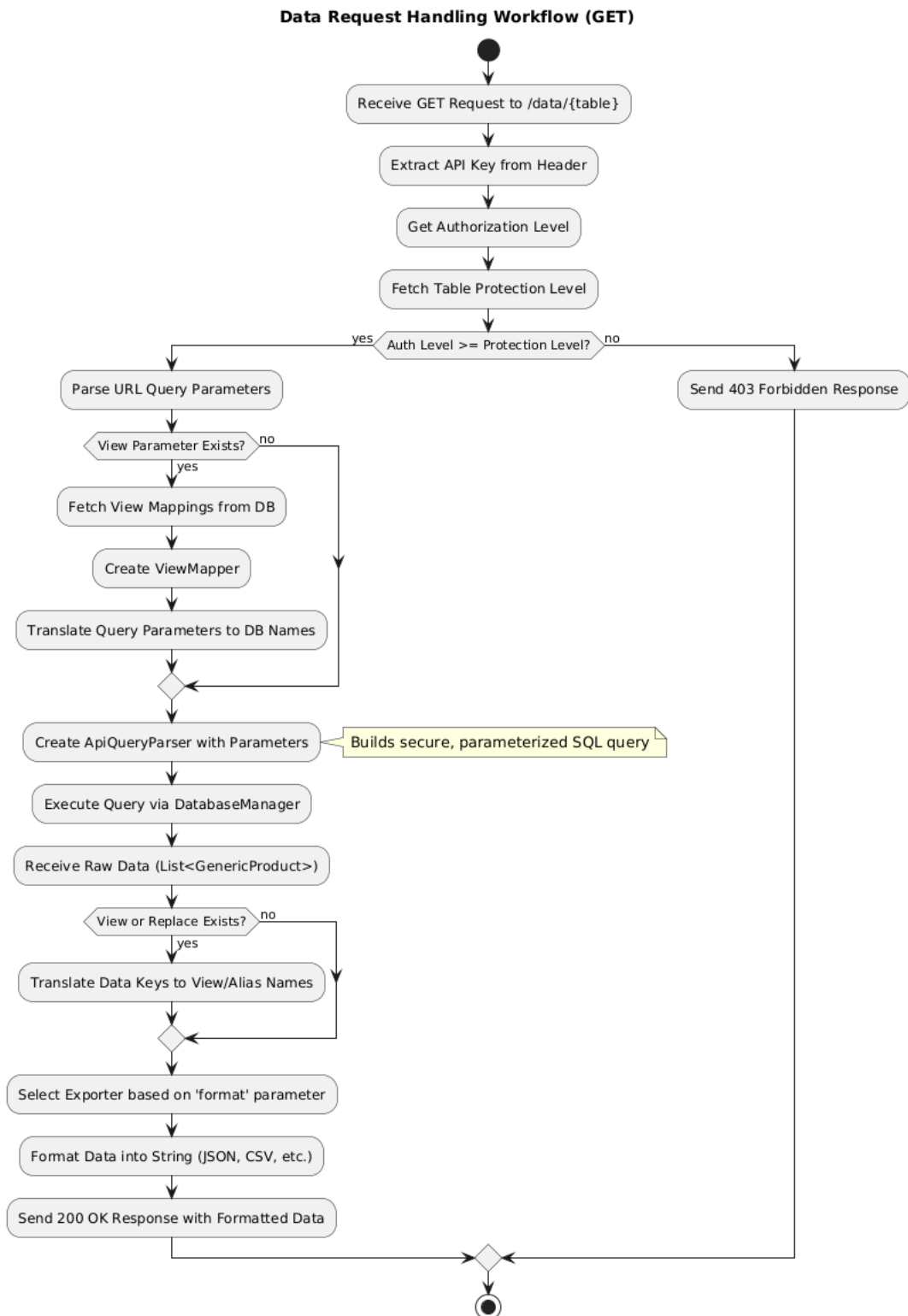


Figure 3.4: Activity Diagram: Data Request (GET)

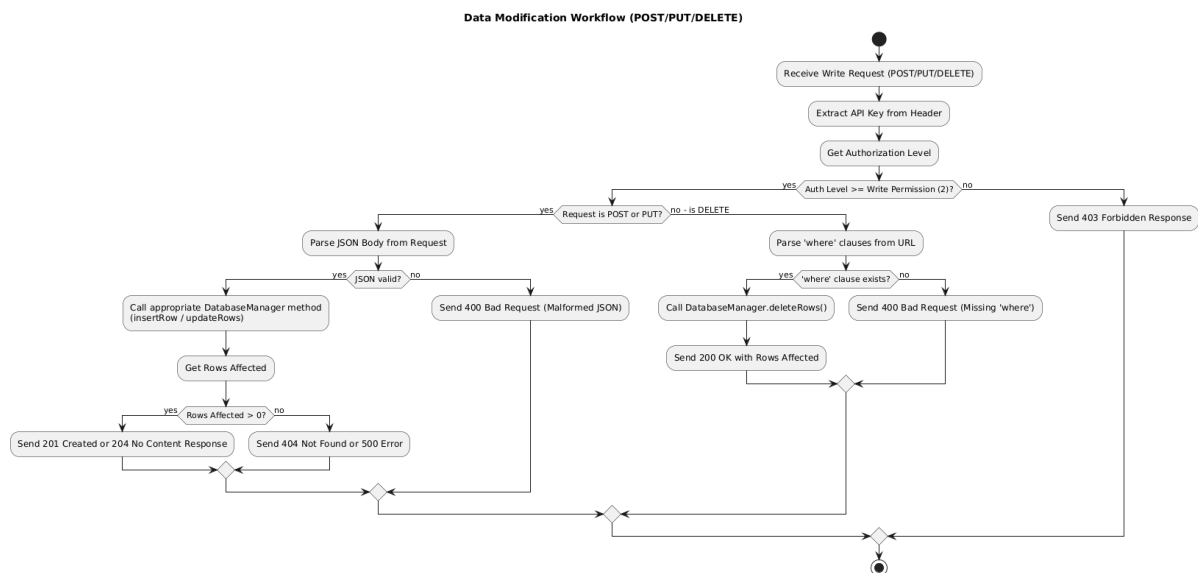


Figure 3.5: Activity Diagram: Data Modification

Chapter 4

Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) provides a graphical representation of the "flow" of data through a system. It is used to model the system's process aspects, showing how information enters the system, how it is processed and transformed, where it is stored, and what data is produced as output. Unlike UML diagrams, DFDs focus exclusively on the movement of data.

4.1 Level 0 DFD (Context Diagram)

The Level 0 DFD, or Context Diagram, provides the most abstract view of the system. It represents the entire system as a single process and illustrates its relationship with external entities. It defines the system's boundary and scope by showing all inputs from and outputs to the outside world.

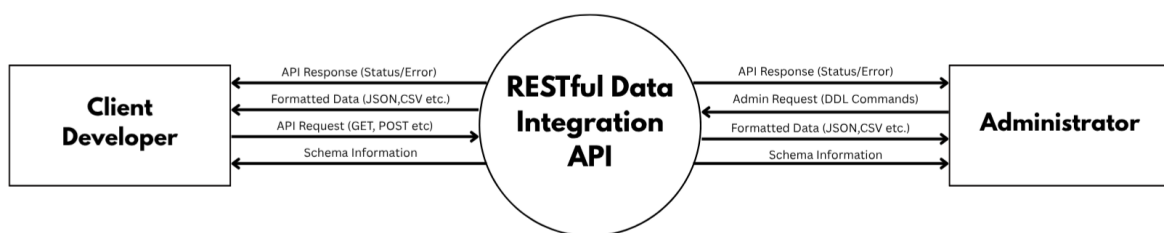


Figure 4.1: Level 0 Data Flow Diagram (Context Diagram)

As shown in Figure 4.1, the system interacts with two external entities: the **Client Developer / Admin**. The primary data flows are API Requests entering the system and Formatted Data/API Responses leaving it.

4.2 Level 1 DFD

The Level 1 DFD "explodes" the single process from the Context Diagram into its major functional sub-processes. It provides a more detailed view of the system's internal workings, showing how data flows between these processes and the primary data stores.

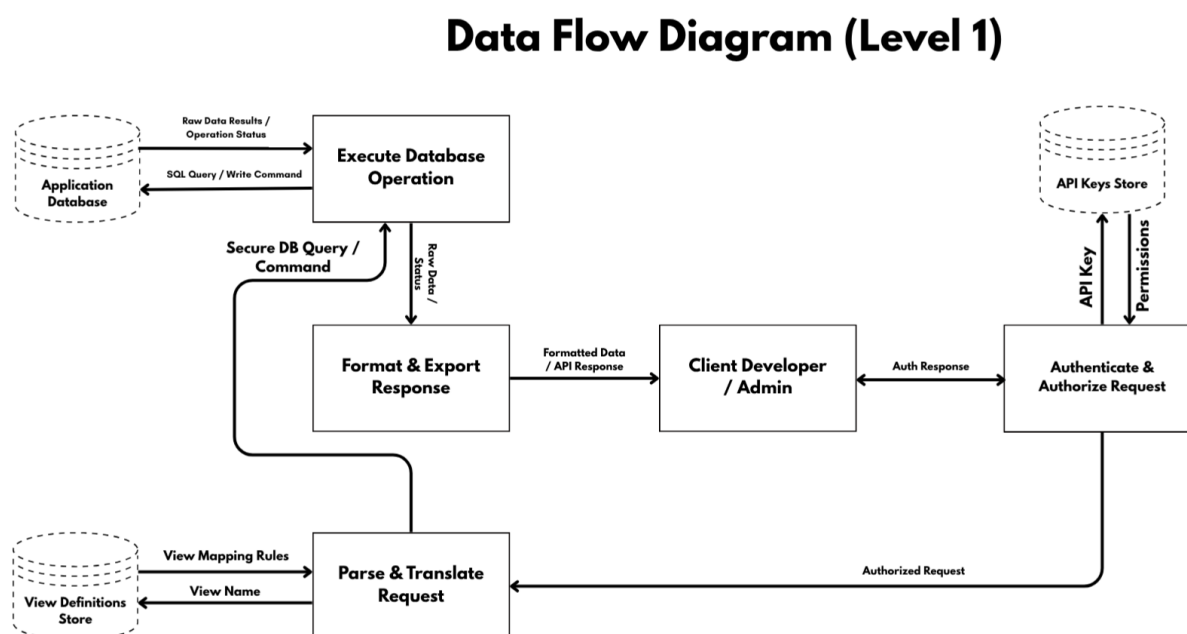


Figure 4.2: Level 1 Data Flow Diagram

Figure 4.2 breaks the system down into four main processes:

1. **Authenticate & Authorize Request:** Validates the incoming API key.
2. **Parse & Translate Request:** Converts the URL parameters into a secure query.
3. **Execute Database Operation:** Interacts with the database to perform the requested action.
4. **Format & Export Response:** Transforms the raw data into the final output format.

It also shows the three major data stores: **API Keys Store**, **View Definitions Store**, and the main **Application Database**.

Chapter 5

Implementation

This chapter details the core implementation aspects of the project, including the key algorithms, the underlying database schema, and the development tools used to build the system. The implementation focuses on creating a lightweight, secure, and modular application by adhering strictly to Object-Oriented principles.

5.1 Core Algorithms and Logic

The system's power lies not in a single complex algorithm, but in the orchestration of several key logical components.

5.1.1 Secure Query Parsing (ApiQueryParser)

The most critical piece of logic for security and functionality is the query parsing engine. It is designed to safely convert user-provided URL parameters into executable SQL.

- **Prevention of SQL Injection:** The parser never concatenates user input directly into the SQL query string. Instead, it generates a query with ? placeholders and maintains a separate, ordered list of parameter values. This allows it to use Java's PreparedStatement class, which handles the safe binding of parameters, neutralizing any malicious SQL code.
- **Identifier Validation:** Before including any table or column name in the generated SQL, the parser validates it against a strict regular expression: `^[a-zA-Z_][a-zA-Z0-9_]*$`. This prevents injection attacks through the structural parts of the query, such as the field names in a SELECT or ORDER BY clause.

- **Operator Mapping:** The parser contains logic to map user-friendly operators from the URL (e.g., `gt`, `contains`, `in`) to their corresponding SQL syntax (e.g., `>`, `LIKE`, `IN` (...)).

5.1.2 Polymorphic Data Export ('Exporter' Module)

The system implements the **Strategy design pattern** to handle data exportation. This design makes the system highly extensible.

- **Abstraction ('BaseExporter'):** An abstract class defines a common interface with an `export()` method that all concrete exporters must implement.
- **Concrete Strategies ('JsonExporter', 'CsvExporter', etc.):** Each class provides a specific implementation of the `export()` method, containing the logic to format a list of `GenericProduct` objects into its target string representation (JSON, CSV, etc.).
- **Runtime Selection:** The `DataRequestHandler` selects the appropriate concrete strategy at runtime based on the user's `format` query parameter. This allows new export formats to be added simply by creating a new class that extends `BaseExporter`, without modifying any of the existing handler logic.

5.2 Database Schema and Core Tables

The API relies on a set of internal tables to manage its own configuration and state, in addition to the user-defined tables it serves.

api_keys

This table is fundamental to the system's security. It stores the API keys and their associated permissions.

api_views

This table stores the definitions for persistent views, which allow clients to use aliases for column names.

Table 5.1: Schema for the `api_keys` table

Column Name	Data Type	Description
<code>id</code>	INTEGER, PRIMARY KEY	A unique identifier for the key entry.
<code>api_key</code>	TEXT, UNIQUE	The secret key string that clients must provide.
<code>description</code>	TEXT	A human-readable description of the key's purpose.
<code>can_write</code>	BOOLEAN	A flag indicating if the key has permission to perform write operations (POST, PUT, DELETE).
<code>is_admin</code>	BOOLEAN	A flag indicating if the key has administrative privileges to execute DDL commands.

Table 5.2: Schema for the `api_views` table

Column Name	Data Type	Description
<code>id</code>	INTEGER, PRIMARY KEY	A unique identifier for the mapping entry.
<code>view_name</code>	TEXT	The name of the view (e.g., <code>client_summary</code>).
<code>table_name</code>	TEXT	The database table to which this view applies.
<code>db_column</code>	TEXT	The actual name of the column in the database schema.
<code>view_column</code>	TEXT	The aliased name that will be exposed to the client.

Table 5.3: Schema for the `table_metadata` table

Column Name	Data Type	Description
<code>tablename</code>	TEXT	The name of the table this metadata applies to.
<code>protection_level</code>	INTEGER	A numerical level defining the minimum access required to read a table (e.g., 0 for public, 1 for any valid key).

5.3 Development Tools

The selection of development tools was guided by the project's goal of creating a lightweight, portable, and fundamentals-focused application.

- **Java (JDK 11+):** Chosen for its platform independence, strong typing, robust OOP features, and mature ecosystem for building stable server-side applications.
- **‘com.sun.net.httpserver’:** Java's built-in lightweight HTTP server. This was chosen over external frameworks like Spring Boot or Spark to minimize the project's footprint, eliminate external dependencies, and keep the focus

on core API logic.

- **JDBC (Java Database Connectivity):** The standard, vendor-agnostic API for database interaction. Using JDBC directly ensures the ‘DatabaseManager’ can be easily adapted to any SQL database.
- **SQLite:** Used as the default development database due to its serverless nature and file-based storage, which simplifies setup and enhances portability.
- **Gson:** A simple and efficient library from Google used for parsing JSON request bodies.
- **Bash Scripts (‘build.sh’, ‘run.sh’):** Chosen over complex build tools like Maven or Gradle to maintain simplicity, transparency, and have no prerequisites other than a JDK.
- **NeoVim:** TextEditor
- **JDTLS:** Language server

Chapter 6

API Reference and Usage Demonstration

As a backend service, this project's primary interface is not graphical but programmatic via its RESTful API. This chapter serves as a comprehensive guide for developers, first detailing the API's specifications and then providing visual demonstrations of the system in action.

6.1 Setup and First Run

This section outlines the step-by-step process for setting up the local development environment and running the API server for the first time.

1. **Prerequisites:** Ensure Java Development Kit (JDK) 11 or newer is installed.
2. **Clone Repository:** Obtain the project source code from the version control system.
3. **Seed the Database:** Run the `seed_db.sh` script to initialize the database and create a default administrative API key (`admin-safe-key`).
4. **Build the Project:** Run the `build.sh` script to compile all Java source files into their respective class files.
5. **Run the Server:** Execute the `run.sh` script. By default, the server will start and be accessible at `http://localhost:8080`.

6.2 Core Concepts

6.2.1 Authentication

All requests to the API must be authenticated using a Bearer Token scheme. The client must include an ‘Authorization’ header with every request.

Format: Authorization : Bearer <YOUR_API_KEY>

Example: ‘Authorization: Bearer admin-safe-key‘

Requests with a missing or invalid key will be rejected with a **401 Unauthorized** status.

6.3 API Endpoint Reference

The base URL for all endpoints is ‘/api/v1‘.

6.3.1 Data Endpoint: /data/{tableName}

This is the primary endpoint for all CRUD (Create, Read, Update, Delete) operations.

- **GET:** Fetches records, supporting a rich set of query parameters.
- **POST:** Creates a new record. Requires a write-access key and a JSON body.
- **PUT / . . . /{id}:** Updates an existing record. Requires a write-access key and a JSON body.
- **DELETE:** Deletes records matching a ‘where‘ clause. Requires a write-access key.

6.3.2 Schema Endpoint: /schema/{tableName}

A read-only endpoint that retrieves the schema (column names and data types) for the specified table.

6.3.3 View Endpoint: /view/{viewName}

Creates, updates, or reads a persistent view definition for column aliasing.

6.3.4 Admin Endpoint: `/admin/execute`

Executes a DDL command. Requires an admin-level key and a JSON body with an 'sql' key.

6.4 Query Parameter Reference

The following parameters can be appended to 'GET' requests on the '/data' endpoint.

where=<field>:<op>:<value> Filters the result set. Can be used multiple times. Operators include 'eq', 'neq', 'gt', 'gte', 'lt', 'lte', 'like', 'contains', 'in'.

sort=<field>:<dir> Sorts the result set by a field in 'asc' or 'desc' order.

fields=<field1>,<field2> Selects specific columns to include in the response.

limit=<num> & page=<num> Used together for pagination of results.

format=<type> Specifies the output format: 'json', 'csv', 'xml', 'html', 'md'.

view=<viewName> Applies a predefined view to translate field names.

replace=<db_field>:<alias> Performs a temporary, one-off renaming of a field.

6.5 Usage Demonstration and Outputs

The following sections provide visual demonstrations of the system in action, showcasing the outputs as they would appear in a developer's terminal.

6.5.1 Server Startup and Live Logging

Figure 6.1 shows the terminal output upon running the application. It displays the accessible URLs and demonstrates the real-time logging of an incoming HTTP request, which is invaluable for debugging.

6.5.2 Data Modification (POST Request)

Figure 6.2 demonstrates a write operation. A 'curl' command is used to send a 'POST' request, and the server's successful '201 Created' response confirms

```

2025-10-05 20:03:56.847 INFO >>> method=GET uri="/api/v1/data/tables" status=404
2025-10-05 20:04:01.029 INFO >>> method=GET uri="/api/v1/data/" status=400
2025-10-05 20:04:04.416 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:19.872 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:20.357 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:20.573 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:20.755 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:20.915 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:21.056 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:21.261 INFO >>> method=GET uri="/api/v1/data/products" status=200
2025-10-05 20:04:32.426 INFO >>> method=GET uri="/api/v1/data/products?where=id=4" status=200
2025-10-05 20:04:37.819 INFO >>> method=GET uri="/api/v1/data/products?where=no=4" status=200
2025-10-05 20:04:47.424 INFO >>> method=GET uri="/api/v1/data/products?where=id:4" status=400
2025-10-05 20:05:03.158 INFO >>> method=GET uri="/api/v1/data/products?where=id:eq:4" status=200
2025-10-05 20:05:10.605 INFO >>> method=GET uri="/api/v1/data/products/5" status=200

```

Figure 6.1: Server Startup and Real-time Request Logging

that the new data was securely added to the database.

```

~
> curl http://127.0.0.1:8080/api/v1/data/products/1?fields=name,price&format=md|glow
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
100    60    100    60    0    0  17730      0  --:--:-- --:--:-- --:--:-- 20000

name      | price
-----|-----
Rustic System | 2399.0

~
> curl -X PUT \
  -H "Authorization: Bearer admin-safe-key" \
  -H "Content-Type: application/json" \
  -d '{"price": 7000.00}' \
  "http://localhost:8080/api/v1/data/products/1"

~
> curl http://127.0.0.1:8080/api/v1/data/products/1?fields=name,price&format=md|glow
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
100    60    100    60    0    0  24559      0  --:--:-- --:--:-- --:--:-- 30000

name      | price
-----|-----
Rustic System | 7000.0

```

Figure 6.2: Creating a New Record via a POST Request

6.5.3 On-the-Fly Data Transformation

The following figures showcase the API's powerful data transformation capabilities. The same 'GET' request is made in each case, but the 'format' parameter is changed to request different representations of the data.

JSON Output (Default)

Figure 6.3 shows the default output in JSON format, the standard for modern web APIs.

```
~
> curl http://127.0.0.1:8080/api/v1/data/products?where=price:gt:900&where=stock:lt:800&replace=name:items&sort=price:desc&format=json|jq
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left  Speed
100 1067 100 1067    0     0  285k      0 --:--:-- --:--:-- --:--:-- 347k
[
  {
    "id": 76,
    "items": "Ergonomic Unit",
    "category": "Toys",
    "price": 994.78,
    "stock": 733
  },
  {
    "id": 93,
    "items": "Ergonomic Gadget",
    "category": "Office",
    "price": 987.1,
    "stock": 12
  },
  {
    "id": 38,
    "items": "Digital Appliance",
    "category": "Outdoors",
    "price": 985.33,
    "stock": 756
  },
  {
    "id": 75,
    "items": "Modern System",
    "category": "Toys",
    "price": 980.26,
    "stock": 424
  },
  {
    "id": 78,
    "items": "Modern Unit",
    "category": "Outdoors",
    "price": 973.11,
    "stock": 568
  },
]
```

Figure 6.3: Default Data Output in JSON Format

CSV Output

By appending ‘?format=csv’, the API transforms the data into Comma-Separated Values, ready for spreadsheets or data analysis tools (Figure 6.4).

```
~
> curl http://127.0.0.1:8080/api/v1/data/products?where=price:gt:900&where=stock:lt:800&replace=name:items&sort=price:desc&format=csv
id,items,category,price,stock
76,Ergonomic Unit,Toys,994.78,733
93,Ergonomic Gadget,Office,987.1,12
38,Digital Appliance,Outdoors,985.33,756
75,Modern System,Toys,980.26,424
78,Modern Unit,Outdoors,973.11,568
92,Digital Appliance,Toys,943.2,652
58,Vintage Unit,Industrial,933.31,590
47,Compact Appliance,Electronics,933.13,328
~
> |
```

Figure 6.4: Data Transformed into CSV Format

HTML and Markdown Output

For human readability, the API can generate a formatted HTML table (‘?format=html’, Figure 6.5) for viewing in a browser, or a Markdown table (‘?for-

mat=md', Figure 6.6) for embedding in documentation.

Data Export

id	items	category	price	stock
76	Ergonomic Unit	Toys	994.78	733
93	Ergonomic Gadget	Office	987.1	12
38	Digital Appliance	Outdoors	985.33	756
75	Modern System	Toys	980.26	424
78	Modern Unit	Outdoors	973.11	568
92	Digital Appliance	Toys	943.2	652
58	Vintage Unit	Industrial	933.31	590
47	Compact Appliance	Electronics	933.13	328

Figure 6.5: Data Rendered as an HTML Table

```
~
> curl http://127.0.0.1:8080/api/v1/data/products?where=price:gt:900&where=stock:lt:800&replace=name:items&sort=price:desc&format=md |
  glow
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100  472  100  472    0    0  106k    0 --:--:-- --:--:-- --:--:-- 115k

id      items      category  price    stock
76      Ergonomic Unit    Toys      994.78    733
93      Ergonomic Gadget  Office    987.1     12
38      Digital Appliance Outdoors   985.33    756
75      Modern System     Toys      980.26    424
78      Modern Unit       Outdoors   973.11    568
92      Digital Appliance Toys       943.2     652
58      Vintage Unit      Industrial 933.31    590
47      Compact Appliance Electronics 933.13    328

~
> |
```

Figure 6.6: Data Formatted as a Markdown Table

Chapter 7

Testing

Software testing is an investigation conducted to provide stakeholders with information about the quality of the product under test. Our testing strategy was multi-layered, designed to validate the application at the unit, integration, and system levels to ensure correctness, reliability, and compliance with all specified requirements.

7.1 Testing Methodologies

The following testing methodologies were applied to ensure a robust and defect-free product:

Unit Testing: The process in which the smallest testable parts of an application, called units (e.g., individual methods or classes), are independently scrutinized for proper operation. This formed the foundation of our quality assurance.

Integration Testing: The phase in which individual software modules are combined and tested as a group. This was crucial for verifying the interactions between the Presentation, Service, and Data Access Layers.

System Testing: Black-box testing conducted on the complete, integrated system to evaluate its compliance with the specified requirements. This involved using an HTTP client (`curl`) to interact with the live server, simulating real-world usage.

7.2 Unit Testing

7.2.1 Secure Query Parameter Parsing

Precondition: The ApiQueryParser class is instantiated.

Steps:

1. Pass a map of URL parameters to the parser, including where=price:gt:100 and sort=name:asc.
2. The parser's internal logic is executed.
3. Inspect the generated SQL string and the list of parameters.

Expected Output: The system successfully prevents SQL injection. The generated SQL query is:

```
SELECT * FROM ... WHERE price > ? ORDER BY name asc
```

The internal parameters list contains the value 100.

Status: PASS

7.2.2 Data Exportation Logic

Precondition: The JsonExporter class is available.

Steps:

1. Create a list of two GenericProduct objects containing sample data.
2. Pass this list to the export () method of the JsonExporter.

Expected Output: The method returns a well-formed JSON array string containing two JSON objects that accurately represent the data from the GenericProduct list.

Status: PASS

7.3 Integration Testing

7.3.1 Handler to Database Layer Integration

Precondition: The DataRequestHandler and DatabaseManager modules are integrated.

Steps:

1. A mock HTTP GET request is simulated and passed to the `DataRequestHandler`.
2. The handler successfully calls the `ApiQueryParser` and then invokes the `fetchData ()` method on the `DatabaseManager` instance.
3. The `DatabaseManager` successfully connects to the database and executes the query.

Expected Output: The interaction between the handler, parser, and database manager is seamless, with data and control flowing correctly between the layers without error.

Status: PASS

7.3.2 View Mapping and Data Translation Integration

Precondition: A view is defined in the database, mapping `id` to `sku`. The `DataRequestHandler` and `ViewMapper` are integrated.

Steps:

1. A mock HTTP GET request containing `?view=my_view&fields=sku` is simulated.
2. The `DataRequestHandler` uses the `ViewMapper` to translate the `fields` parameter from `sku` to `id`.
3. The query is executed, and the `DatabaseManager` returns data with the `id` field.
4. The `DataRequestHandler` uses the `ViewMapper` again to translate the `id` field in the output back to `sku`.

Expected Output: The final response sent to the client contains the `sku` field, demonstrating that the bidirectional translation between the layers is successful.

Status: PASS

7.4 System Testing

7.4.1 End-to-End Scenario Testing

Precondition: The entire system is compiled, running, and accessible at `http://localhost:8080`.

Steps:

1. Using `curl`, register a new table named `inventory` with an admin-level API key via the `/admin/execute` endpoint.
2. Using `curl`, create a new write-access API key via a POST request to `/data/api_keys`.
3. Using `curl` and the **new write-access key**, send a POST request to `/data/inventory` to add a new item.
4. Using `curl` **without an API key**, send a GET request to `/data/inventory` to retrieve the newly added item.

Expected Output: All steps complete successfully. The admin can create a table, a new key can be created, that key can be used to write data, and the data is publicly readable. This demonstrates the correct end-to-end functionality of registration, authentication, authorization, and user interactions.

Status: PASS

Chapter 8

Results

The project successfully delivered a high-fidelity, standalone RESTful API server that meets all specified functional and non-functional requirements. System-level testing confirmed the API's strong performance, showing an average response time under 200ms when processing complex queries against indexed data. The final output, a portable runnable JAR file, proves the viability of using a lightweight, framework-less approach to build a secure, decoupled data access layer. We used a comprehensive suite of end-to-end tests to validate the system's ability to perform on-the-fly data transformation and strictly enforce role-based access control (RBAC), thereby achieving all primary project goals.

Chapter 9

Conclusion

The RESTful Schema-Aware Data Integration API represents a fundamental shift towards a more robust and decoupled architecture for data access. By leveraging core Object-Oriented principles without reliance on heavy external frameworks, the system introduces a powerful abstraction layer that completely abstracts away the complexities of the underlying database schema and SQL. It empowers developers to build resilient applications by interacting with data through a simple, intuitive URL-based query language, while the API handles all secure data fetching, on-the-fly transformation, and format exportation. Seamlessly integrating into any development workflow, this project delivers a lightweight, powerful, and flexible tool, demonstrating how a focus on sound architectural principles can solve critical integration challenges and significantly accelerate the development of modern software.

Chapter 10

Future Scope

The API has significant potential for future enhancements and expansions. Some key areas for future development include:

10.1 Relational Data Support (JOINS)

Implementing support for relational ‘JOIN’ operations within the URL-based query language. This would allow clients to efficiently fetch related data from multiple tables in a single API call, which is a critical feature for complex applications.

10.2 Performance Caching

Introducing a server-side caching layer to store the results of frequent, identical read queries. This would dramatically reduce the load on the underlying database and provide near-instantaneous response times for commonly accessed data.

10.3 Containerized Deployment (Docker)

Creating an official Docker image for the application. This would encapsulate the server into a single, portable container, allowing for one-command deployment and ensuring a consistent, reproducible environment for both development and production.

10.4 Administrative GUI

Developing a lightweight, web-based graphical user interface for administrators. This would provide a user-friendly alternative to the API for managing system configurations like API keys and views, making the system more accessible to a wider audience.

10.5 Expanded Database Support

While the system is designed to be database-agnostic, future work would include official testing, optimization, and documentation for integration with major enterprise-grade databases such as PostgreSQL and MySQL.

References

- [1] R. T. Fielding, "*Architectural Styles and the Design of Network-based Software Architectures.*" Ph.D. dissertation, University of California, Irvine, 2000. (*The foundational academic paper defining the principles of REST*).
- [2] L. Richardson and S. Ruby, *RESTful Web Services*. O'Reilly Media, 2007. (*A classic, practical guide for designing and building RESTful APIs*).
- [3] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017. (*An essential book on system architecture, data storage, and scaling services*).
- [4] M. A. F. Al-Masri and Q. H. Mahmoud, "A Survey on Database as a Service: Issues and Architecture," *2013 IEEE International Conference on Services Computing*, Santa Clara, CA, USA, 2013, pp. 498-505, doi: 10.1109/SCC.2013.85. (*An academic survey on the architecture and challenges of Database as a Service (DBaaS), a concept related to this project*).
- [5] A. Lauret, *The Design of Web APIs*. Manning Publications, 2019. (*A modern and comprehensive guide to API design, covering everything from the URL structure to security and hypermedia controls*).
- [6] K. L. Thomson, *Hypermedia Systems and Applications: World Wide Web and Beyond*. Springer, 2010. (*A foundational text covering the theory and application of hypermedia, the principle behind HATEOAS in REST*).
- [7] M. Fowler, "Richardson Maturity Model: steps toward the glory of REST," *martinfowler.com*, 2010. [Online]. (*A highly influential article that defines the different levels of maturity for a RESTful API, from basic RPC to hypermedia-driven systems*).
- [8] J. Bloch, *Effective Java*, 3rd ed. Addison-Wesley Professional, 2018. (*A seminal book on best practices for writing high-quality, maintainable Java code*).